

Plasticity Is All You Need?

A Testable Proposal for Persistent Fast-Weight Adaptation in Neural Architectures

Thomas Nauheimer

Cognitive Systems Research • Nierstein, Germany • nauheimer.t@gmail.com

Abstract

We investigate a frozen linear transformation augmented with persistent low-rank fast weights. Unlike external sidecar architectures, the supplied PlasticLinear adapter participates directly in a PyTorch model's forward computation. It applies explicit target-driven delta-rule updates and protects a specified input subspace $\text{span}(Q)$. QR decompositions and a small-core SVD compress updates without explicitly reconstructing the dense fast-weight matrix. Synthetic linear-regression and Llama-style decoder experiments demonstrate adaptation, preservation of protected inputs, unchanged base weights, and reliable state serialization. These experiments establish implementation correctness in a testable, reproducible framework for continual fast-weight adaptation.

1. Relationship to Existing Work

Fast weights and trainable plasticity have established theoretical precedents in neuromorphic computing. Ba et al. (2016) explored temporary fast-weight memory for recurrent architectures; Miconi et al. (2018) optimized differentiable plasticity through an outer optimization loop; and Kirkpatrick et al. (2017) employed Fisher-based parameter importance (EWC) to mitigate catastrophic forgetting. In contrast to external retrieval-augmented generation (RAG) which maintains state outside model representations, fast-weight plasticity internalizes recent associations directly into the functional forward mapping.

2. Implemented Model and Mathematical Formulation

For row-batched inputs $X \in \mathbb{R}^{N \times d_{\text{in}}}$, the forward mapping is defined as:

$$A(t) = U(t) V(t)^T$$

$$Y = X W_{\text{slow}}^T + b + \gamma (X V) U^T$$

where W_{slow} is the frozen base parameter matrix, $U \in \mathbb{R}^{d_{\text{out}} \times r}$ and $V \in \mathbb{R}^{d_{\text{in}} \times r}$ are dynamic low-rank factor buffers initialized to zero, and $\gamma \in [0, 1]$ scales adaptation energy. Given target activations T , error residual $E = T - Y$, and orthogonal subspace projector $P = I - Q Q^T$, the candidate update satisfies:

$$A_{\text{cand}} = (1 - \lambda) A + (\eta \gamma / N) E^T X P$$

3. Subspace Protection & Invariance Invariants

Let $Q \in \mathbb{R}^{d_{in} \times k}$ denote an orthonormal basis for protected input features. If $A_0 Q = 0$ and every successive update is right-projected by $P = I - Q Q^T$, then $A_t Q = 0$ holds identically in exact arithmetic. Consequently, the adapter leaves the output of the linear layer strictly invariant for all inputs in $\text{span}(Q)$. Our implementation re-projects right factors following low-rank SVD compression, ensuring that numerical interference remains within machine precision ($\epsilon \approx 10^{-16}$).

4. Low-Rank Compression via Thin QR & Core SVD

Representing candidate rank updates as $L R^T$ with $m = r + N$ columns:

- $L = [\sqrt{(1-\lambda)} U, \sqrt{(\eta \gamma N)} E^T]$
- $R = [\sqrt{(1-\lambda)} P V, \sqrt{(\eta \gamma N)} P X^T]$

Thin QR factorizations $L = Q_L R_L$ and $R = Q_R R_R$ reduce the singular value decomposition to the small core matrix $R_L R_R^T$. Truncating to rank r and enforcing a Frobenius norm cap avoids dense matrix materialization, maintaining strict $O(N r (d_{in} + d_{out}))$ runtime efficiency.

5. Empirical Validation & Reproducibility

We evaluated *PlasticLinear* across five independent random seeds (CPU, float64, 32-to-16 projection, rank 8, 4 protected directions, 8 novel adaptation directions, 160 updates).

Seed	Held-out MSE Before	Held-out MSE After	Max Protected Change	Status
0	0.254999	2.74e-10	3.47e-17	Verified
1	0.291153	3.13e-10	2.78e-17	Verified
2	0.227038	2.44e-10	2.78e-17	Verified
3	0.413457	4.44e-10	9.02e-17	Verified
4	0.501408	5.38e-10	3.47e-17	Verified

In all seeds, base weights remained bit-identical, state_dict serialization roundtrips were exact, and protected subspace error was bounded by floating-point noise ($\leq 10^{-16}$).

6. Conclusion & Availability

Fast-weight plasticity provides a viable, computationally efficient mechanism for real-time model adaptation. The complete PyTorch implementation, unit test suite, and interactive Transformer demonstration are released as open-source software under the MIT License at: <https://github.com/nauheimervater/plastic-transformer>.